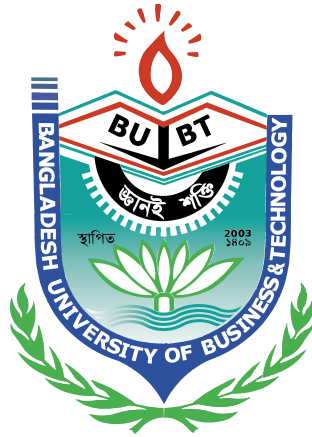


BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY (BUBT)

DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING



PROJECT PROPOSAL

On

Web-Based Multi-Accessible IDE for Team Projects

Submission Date: 7th July, 2026

Submitted By:

Name: Md. Emam Zafor Saadik

ID: 22235103581

Intake/Section: 51/02

Name: Samia Sultana

ID: 22235103292

Intake/Section: 51/02

Name: Halima Tus Sadia

ID: 22235103557

Intake/Section: 51/02

Submitted To:

Shampa Banik

Lecturer, Dept. of CSE

Bangladesh University of Business and
Technology (BUBT)

1 Needs / Problems

In the landscape of computer science education and remote software development, distributed teams face critical point-of-friction workflow bottlenecks. The primary problems this system addresses are:

- **Environment Mismatch (“Works on My Machine” Syndrome):** Team members waste critical development hours configuring localized machine environments, operating system variables, and mismatched software runtimes instead of writing actual project code.
- **Complexity of Traditional Version Control:** Managing Git branches and manual merge conflicts frequently overwhelms student developers, causing accidental code overwrites or broken repositories during tight academic submission deadlines.
- **Hardware and Resource Constraints:** Resource-heavy desktop IDEs demand high system CPU and RAM allocations, slowing performance and restricting smooth collaboration for teammates using lower-specification laptops.

2 Goals / Objectives

The core objective of this project is to construct a browser-accessible, centralized cloud workspace wrapper that completely eliminates local setup friction and streamlines folder tracking for academic development teams.

The explicit targets are:

- **Lightweight Container Environment:** To build a lightweight, secure development container environment operating fully inside web browsers with absolute zero local configuration requirements.
- **Atomic Synchronization Architecture:** To establish an atomic “Save-and-Sync” data pattern that isolates and centralizes repository files safely inside a primary cloud database to eliminate conflicting code overwrites.
- **Unified Collaborative Dashboard:** To lower entry barriers for collaborative programming by combining structural workspace dashboards, nested file explorers, and local frontend preview engines into a single accessible interface.

3 Existing System Analysis

To establish contextual positioning, we evaluate two prominent cloud workspace systems currently operating in the market:

- **Replit [1]:** A highly scalable cloud-based development studio. While globally operational, its business model heavily limits free-tier operations, restricting collaborative computational processing allowances and locking private multi-user team spaces behind premium paywalls.

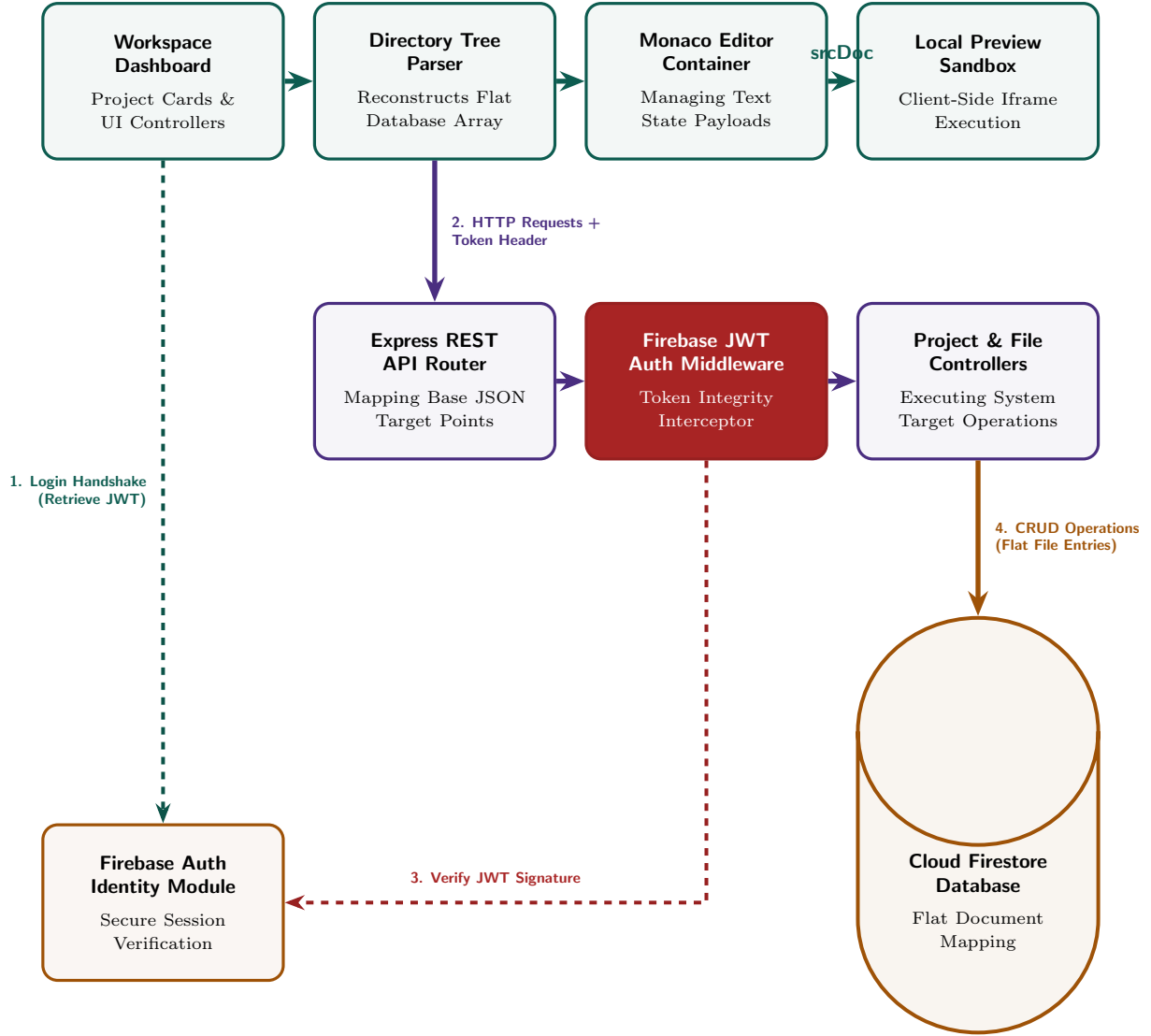


Figure 1: Full-Stack REST System Architecture Component Flow Diagram.

- **CodeSandbox [2]:** Optimized heavily for advanced web-application frameworks. However, its structural layout presents significant configuration bloat for basic academic tasks, containing multi-tier container options and deployment tools that create a steep learning curve for entry-level engineering squads.

3.1 Shortcomings and Our Distinct Value Proposition

The core limitation of commercial tools is their structural emphasis on enterprise DevOps configurations. They treat small student project teams like production engineering divisions, forcing dense setup choices onto basic development environments.

Our proposed project resolves this imbalance. By shifting away from complex virtual machines, our platform focuses purely on being a streamlined, lightweight multi-accessible project repository hub. It strips away operational overhead, optimizing user spaces purely for file arrangement, centralized database saving, and clean team resource sharing.

4 Features / Scopes

To protect project delivery constraints within our 2-month semester schedule, the system boundaries are restricted to the following functional core pillars:

- **User Authentication Module:** Implements registration, login verification, and session persistence powered via Firebase Client Web Auth layers.
- **Project Space CRUD Engine:** Interactive dashboards allowing authenticated developers to initialize new workspace containers, list projects they hold access rights to, alter project metadata, and execute deletions.
- **Access Invitation Handler:** Project managers can inject a teammate’s registered email address to append their permissions into a project’s access control array, facilitating multi-user entry access.
- **Flat Directory File Tree System:** A left-hand navigation explorer transforming flat database rows into visual nested folders, supporting on-the-fly text file instantiations and local file uploads.
- **Monaco Code Container Wrapper:** Structural embedding of the Microsoft Monaco Editor component [3] inside React to enable multi-language syntax highlighting, autocomplete assistance, and an explicit manual “Save Changes” action link.
- **Unsaved Status Flag Indicators:** Local event trackers that dynamically trigger a visual warning indicator icon on open file tabs when text edits have been typed but not safely committed to the database.
- **Client-Side HTML Web Preview:** A split-screen rendering frame utilizing local browser iframe `srcDoc` attributes to assemble and run static HTML, CSS, and JS web views instantly without drawing backend server processing overhead.

5 Development Model

The project workflow will utilize the **Incremental and Iterative Development Model**. Given our rapid 2-month university timeline, linear development presents too high a risk of non-delivery.

The iterative methodology mitigates this hazard by breaking development into isolated system lifecycles:

- **Iteration 1 (Weeks 1–3):** Building Express core server routing architectures, configuring initial Firestore metadata schemas, and verifying basic layout components.
- **Iteration 2 (Weeks 4–6):** Embedding the Monaco editor wrapper, establishing the flat-directory reconstruction handlers, and mounting user access array verifications.
- **Iteration 3 (Weeks 7–8):** Comprehensive validation testing across layout break-points, optimization of database calls, and final presentation preparation.

6 Feasibility Analysis & Possible Risks

6.1 3-Factor Feasibility Analysis

- **Technical Feasibility:** High. The combination of React, Node.js, and Express is exceptionally well-documented across open-source communities, and the web Monaco component provides industry-standard workspace stability [3].
- **Operational Feasibility:** High. Because the execution engine runs natively inside standard client web browsers, end-users face zero operating system constraints, permission dependencies, or runtime installation barriers.
- **Economic Feasibility:** High. By framing the architecture entirely around lightweight plaintext operations, the product operates wholly inside the free quotas of the Firebase Spark plan tier [4], demanding absolute zero development budget (\$0 / BDT 0).

6.2 Project Risks and Defensive Design Mitigation

- **Risk (Database Read Limit Exhaustion):** Faulty React hooks or improperly structured loops could accidentally trigger infinite, recursive data fetching, rapidly burning the daily free Firebase allotment of 50,000 reads and locking down the live demonstration.
- **Mitigation:** We explicitly reject real-time continuous socket or streaming listeners. The database interactions are bound to manual HTTP REST triggers, running reads and writes strictly when users intentionally select a file tab or click the “Save Changes” action.

7 Requirements

- **Hardware Configurations:** Core i3 system workstation or higher, minimum 8 GB System RAM, high-speed internet connectivity.
- **Core Languages:** JavaScript (ECMAScript 6+), HTML5, CSS3.
- **Frontend Libraries & Frameworks:** React.js Engine, Monaco Editor Browser SDK (@monaco-editor/react) [3].
- **Backend Architecture Platforms:** Node.js Runtime with the Express.js Application Routing Framework.
- **Database & Security Engines:** Cloud Firestore (Flat Schema Design Layout) paired with Firebase Auth Management tools [4].
- **Design & Version Control Tools:** Google Stitch wireframing suite, Figma layout platforms, Git client engines, and GitHub organizational repositories.

8 Activity and Time Schedule

The project timeline is structured across 270 engineering man-hours distributed over the 2-month university semester cycle:

Table 1: Project Phase Task Breakdown with Effort Allocation.

Stage	Specific Phase Tasks / Deliverables	Efforts (Man-Hours)
Stage 1	Analysis & Architecture Design	
1.1	Requirement Specifications & Google Stitch Wireframe Mock-ups	14
1.2	Database Schema Structural Modeling & Technical Work Plan Creation	6
Stage 2	Full-Stack Implementation	
2.1	Responsive Client UI Layout Code Creation (Figma → React Components)	45
2.2	Express Server Infrastructure & Firebase Admin SDK Configuration Handshaking	40
2.3	User Authentication Security Middleware JWT Token Verification Hooks	30
2.4	File System CRUD Logic (Create / Upload / Delete Request Controllers)	35
2.5	Monaco Editor Active Value State Synchronization Pipelines	40
2.6	Client-Side Dynamic Iframe Sandboxed Preview Compilation Engine	20
Stage 3	Testing & Quality Assurance	
3.1	Cross-Browser Layout Verification & Validation Test Case Optimization	30
Stage 4	Deployment	
4.1	Project Deployment on Serverless Ecosystems (Render and Vercel Environments)	10
Total Estimated Engineering Lab Efforts:		270 Hours

9 Budget

The workspace leverages open-source development packages and free cloud execution tiers to ensure zero economic dependencies:

Table 2: Project Cost Specification Matrix.

Project Item Description	Operational Specifications / Package Scope	Total Price
Development Software	Community Open-Source Kits (VS Code, Git, React, Node)	\$0 / BDT 0
IDE Layout UI Core Engine	Monaco Editor Core Open-Source Package Ecosystem [3]	\$0 / BDT 0
Database / Server License	Firebase Spark Free Account Service Operational Tier [4]	\$0 / BDT 0
Server App Deployment Hosting	Render & Vercel Base Developer Deploy Tiers	\$0 / BDT 0
Workstation Hardware Assets	Personal Internal Team Computing Hardware Attachments	\$0 / BDT 0
Total Project Budget:	Frictionless Open-Source Framework Utility	\$0 / BDT 0

10 Development Team & Stakeholders

10.1 Team Structure and Responsibility Distribution

To optimize execution safety and prevent operational imbalances, tasks are distinctively partitioned according to core engineering strengths, ensuring fair working distributions:

Table 3: Team Membership Designations and Phase Ownership Profiles.

ID	Name	Designation	Technical Project Responsibilities
01	Md. Emam Zafor Saadik	Full-Stack Developer	Lead Backend Express routing architecture, Firebase Admin SDK integration, JWT authentication middleware, and final deployment pipeline management.
02	Samia Sultana	Frontend UI Engineer	React component library construction, Monaco Editor state binding, responsive layout design implementation via Figma design specifications.
03	Halima Tus Sadia	Database & QA Engineer	Firestore flat schema design modeling, CRUD controller logic, cross-browser validation testing, and technical documentation authoring.

10.2 Project Stakeholders

- **Primary Client / Supervisor:** Shampa Banik, Lecturer, Department of CSE, BUBT — defines academic evaluation criteria and final delivery acceptance.

- **End Users:** Student developer teams enrolled in software engineering coursework who require a zero-setup, browser-accessible collaborative coding workspace.
- **Secondary Stakeholders:** Department of Computer Science and Engineering, BUBT — benefits from student exposure to modern full-stack web development practices and cloud infrastructure tooling.

11 Limitations & Future Works

11.1 11.1 Defined System Limitations (Out of MVP Scope)

To secure strict timeline compliance, the application intentionally limits the following features:

- **No Multi-User Live Cursor Telemetry:** Teammates do not stream real-time typing indicators or location paths. File changes coordinate explicitly on explicit user-driven data mutations.
- **Static Sandbox Constraints:** The local execution preview layout frame handles only languages interpreted natively by browser tab scripts (HTML5, CSS3, vanilla JavaScript).
- **No Server-Side Cloud Shell Pipelines:** The backend does not instantiate deep virtual Linux execution loops or compiler instances for terminal-bound native languages (such as C, C++, or Java packages).
- **Custom Mock-Repository Framework:** Directory trees are logged linearly in Firestore via custom relative file path string definitions rather than running live local Git file trees on the server.

11.2 11.2 Planned Future Implementations

- Integrating authorization hooks for modular Agentic AI Large Language Model APIs (such as Gemini API engines) to read whole file structures and generate automatic error fixes inside the workspace.
- Developing automated version control extensions capable of committing saved project directory snapshots directly onto remote GitHub repository branches via external webhooks.

References

- [1] Replit Inc. *Replit: The Collaborative Browser Based IDE*. 2025. Available at: <https://replit.com>
- [2] CodeSandbox B.V. *CodeSandbox: Online Code Editor and IDE*. 2025. Available at: <https://codesandbox.io>
- [3] Microsoft Corporation. *Monaco Editor: The Code Editor That Powers VS Code*. 2026. Available at: <https://microsoft.github.io/monaco-editor/>

- [4] Google LLC. *Firebase Documentation: Build and Run Apps with Google Infrastructure*. 2026. Available at: <https://firebase.google.com/docs>